

SER216 Final Project Report

Digital Parking Permit System

Student: Bobur Komilov

Course: SER216 - Software Engineering

Date: 2026-05-04

Repository: github.com/Komilov08/SER216_Final_Project_KomilovBobur

1. Introduction

A university wants a digital parking permit system. Students and staff register vehicles, request parking permits, check available parking zones, and report parking issues. Security staff verify permits and update violation records.

This report presents the requirements, workflow, test plan, and defect analysis for the proposed system, along with the GitHub evidence that demonstrates a complete development workflow (branch, commits, pull request, merge).

2. Requirements

2.1 Functional Requirements

FR1 - Vehicle Registration

The system shall allow students and staff to register a vehicle by providing the license plate number, make, model, color, and owner ID. Each user may register up to three vehicles per academic year.

FR2 - Permit Request and Approval

The system shall allow a registered user to request a parking permit by selecting a permit type (student, staff, visitor) and a preferred parking zone. The request shall be reviewed and either approved or rejected by the parking office, and the user shall be notified of the decision by email.

FR3 - Real-Time Zone Availability

The system shall display the live availability (free / occupied / reserved) of every campus parking zone so that users can choose a zone with open spaces before submitting a permit request.

FR4 - Permit Verification by Security

The system shall allow security staff to scan a vehicle's license plate or permit QR code and instantly verify whether the displayed permit is valid, expired, or revoked, and to record any violation directly into the user's record.

2.2 Non-Functional Requirements

NFR1 - Performance

The system shall respond to a permit verification scan in under 2 seconds, even when up to 500 verifications are performed concurrently across campus during peak hours.

NFR2 - Security

All personal data (license plates, owner IDs, payment data) shall be transmitted over HTTPS and stored encrypted at rest using AES-256. Only authenticated parking-office and security staff shall be allowed to view or modify permit records.

3. Workflow Diagram

The diagram below shows the end-to-end workflow of the Digital Parking Permit System: Vehicle Registration -> Permit Request -> Zone Check -> Permit Approval -> Parking Verification.

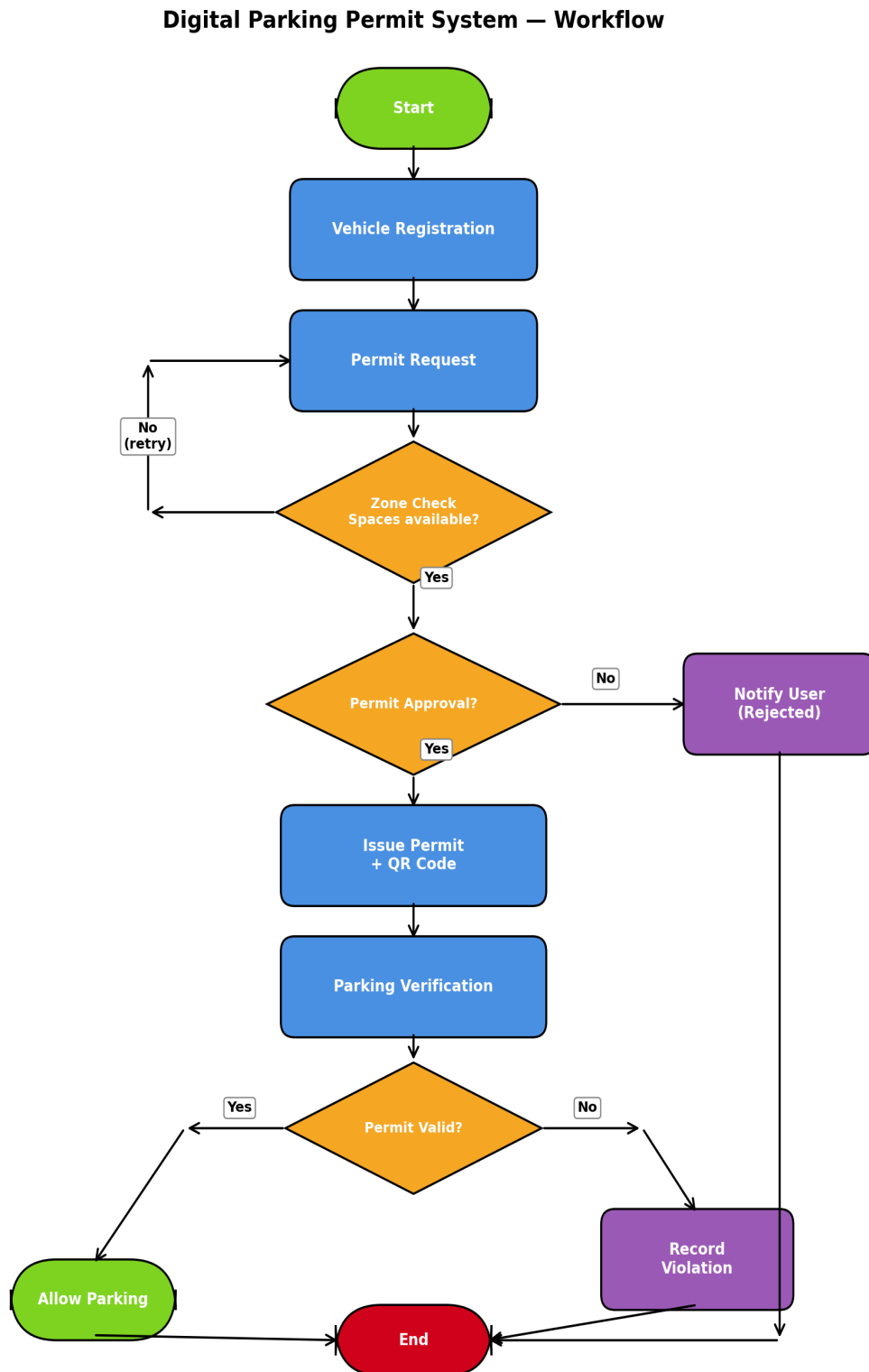


Figure 1. Permit lifecycle from vehicle registration through verification.

4. Test Case Table

Six test cases covering system, acceptance, and regression testing.

Test ID	Feature	Input / Data	Expected Result	Type
TC-01	End-to-end permit issuance	Register vehicle ABC-123, request Student permit for Zone A (4 free spaces), parking office approves	Permit is issued with a unique QR code, user receives approval email, Zone A free count decreases from 4 to 3	System
TC-02	Permit verification under load	Security scans 50 different valid permit QR codes within 10 seconds during peak hours	All 50 scans return Valid in under 2 seconds, no timeouts or duplicates, audit log records all 50 events	System
TC-03	Student requests staff permit	Registered student logs in, selects Staff permit type for Zone B	System rejects the request with a clear message - business rule for permit type is enforced	Acceptance
TC-04	Security reports a violation	Security scans an unregistered plate XYZ-999 and adds note 'No permit displayed'	System creates a violation record marked Pending Review and notifies the parking office	Acceptance
TC-05	Re-verify zone availability	After fixing a defect that miscounted occupied spaces, run the Zone A availability check with 2 cars parked	System reports 2 occupied / correct free count - earlier counting bug stays fixed	Regression
TC-06	Re-verify permit expiry logic	After fixing the timezone bug, scan a permit issued today that expires next semester	System returns Valid regardless of the device timezone - earlier expiry defect stays fixed	Regression

System tests (TC-01, TC-02) check the complete integrated system end-to-end. Acceptance tests (TC-03, TC-04) check whether the system meets user and business expectations. Regression tests (TC-05, TC-06) re-verify previously fixed defects so they do not reappear after later changes.

5. Defect Analysis

Defect: "A valid parking permit is shown as expired during security verification."

Defect category	Functional defect - incorrect business logic. The system returns the wrong status (Expired) for a record whose stored expiry date is still in the future. Not a UI defect (message renders correctly) and not a performance defect (response time is fine); the underlying decision is wrong.
Severity	High. The core promise of the system - telling security whether a permit is valid right now - is broken. A valid customer is wrongly flagged, which can lead to a false violation, customer complaints, and loss of trust. It does not crash the system, so it is not Critical, but it directly blocks a primary use case.
Priority	High (P1). The defect happens during live security verification and is visible to end users in real time. It must be fixed before the next release; a hotfix is appropriate. The manual workaround (overriding the check by hand) is slow and error-prone, so the fix cannot wait.
Possible root cause	Most likely a timezone / date-comparison bug in the verification service: the permit's <code>expires_at</code> is stored in UTC but compared against the security device's local time (or vice versa), so a permit that expires at midnight UTC is reported as expired several hours early in a positive UTC offset. Other possibilities: the expiry field is a string and is being compared lexically rather than as a date; a recent migration shifted dates by one day due to a date vs datetime cast; an old expired record is being served from cache after renewal.
Regression test	Re-run TC-06 after the fix: scan a permit issued today that expires next semester, on devices set to multiple timezones (UTC, UTC+5, UTC-8). Expected result is Valid on every device. Add this test to the automated regression suite so the defect cannot silently return after a future change.

Recommended Fix Workflow

- Reproduce the defect against a copy of production data with at least one near-expiry permit.
- Inspect the verification service's expiry comparison - confirm both sides are timezone-aware datetime values.
- Patch the comparison to always evaluate in UTC, or use a database-level comparison so the database engine handles timezones consistently.
- Add unit tests covering UTC, positive offsets, negative offsets, and the day-boundary edge case.
- Add the regression test (TC-06) to the CI pipeline.
- Deploy as a hotfix and monitor the verification audit log for any recurrence over the next 7 days.

6. GitHub Evidence

The full development workflow is captured in the GitHub repository for this project. The required evidence is summarized below; screenshots are stored in the screenshots/ folder of the repository.

Required Evidence Checklist

- Repository created: SER216_Final_Project_KomilovBobur
- README.md added with project description, tools, structure, and workflow
- Project files uploaded: report PDF, workflow diagram PNG, test cases CSV, defect analysis, screenshots
- Feature branch created: feature/parking-permit-system
- At least 3 meaningful commits made on the feature branch
- Pull request opened from feature branch into main
- Pull request reviewed and merged into main
- Final PDF uploaded to the repository at the root

Repository Link

https://github.com/Komilov08/SER216_Final_Project_KomilovBobur

Commit Examples

- docs: update README with full project description and structure
- feat: add functional and non-functional requirements
- feat: add workflow diagram (Mermaid + PNG)
- feat: add test case table covering system, acceptance, regression
- feat: add defect analysis for expired-permit issue
- docs: add final report PDF

7. Conclusion

The proposed Digital Parking Permit System covers vehicle registration, permit request and approval, real-time zone availability, and security verification with violation tracking. The four functional and two non-functional requirements together set out a system that is correct, fast under load, and secure with personal data.

The test plan combines system, acceptance, and regression cases so that integration, business value, and prior fixes are all verified before a release. The defect analysis for the 'valid permit shown as expired' issue points strongly at a timezone bug in the verification service and prescribes both a hotfix path and a regression test that stops the defect from returning.

All deliverables and supporting evidence are version-controlled on GitHub, with branch, commit, and pull-request history demonstrating a complete software-engineering workflow.